CYBER CHALLENGE.IT

CyberChallenge.IT is the first Italian training program in cybersecurity for high-school and undergraduate students.

Binary Exploitation - 3

Matteo Chiesa, Diego Oliva

Università degli studi di Cagliari

CYBERSECURITY NATIONAL LAB

GPDP GARANTE PER LA PROTEZIONE DEI DATI PERSONALI

https://cyberchallenge.it https://srdnlen.it

Memory Layout

Memory Layout

The Linux kernel manages **V**irtual **M**emory through **pages**, the smallest kernel-indexable memory units.

- Page size on AMD64 is *usually* **0x1000** (4096)
- The lowest **12 bits** (3 nibbles) of *every* memory address are the **offset into the page** that holds the referenced memory
- Address of a page always ends in 0x…**000**
- A memory region of contiguous and homogeneous pages is called “Virtual Memory Area” (**VMA***)
- Memory protection (**R**-**W**-**X**) can only be applied to **whole VMA**s

*try running “cat /proc/self/maps” to see all the process’ VMAs

Memory Layout

Relevant user space **VMA**s are (in order):

- Binary
- Heap (brk): close to .BSS
- Mmap base: shared libraries
- VVAR, VDSO
- Loader
- Stack
- VSYSCALL (legacy, fixed base @ 0xffffffffff600000)

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
            Start                End Perm     Size  Offset File (set vmmap-prefer-relpaths on)
    0x55e34a29a000     0x55e34a29c000 r--p     2000       0 cryptopwn
    0x55e34a29c000     0x55e34a2a1000 r-xp     5000    2000 cryptopwn
    0x55e34a2a1000     0x55e34a2a3000 r--p     2000    7000 cryptopwn
    0x55e34a2a3000     0x55e34a2a4000 r--p     1000    8000 cryptopwn
    0x55e34a2a4000     0x55e34a2a8000 rw-p     4000    9000 cryptopwn
    0x55e37189b000     0x55e3718bc000 rw-p    21000       0 [heap]
    0x7f49ec988000     0x7f49ec98b000 rw-p     3000       0 [anon_7f49ec988]
    0x7f49ec98b000     0x7f49ec9b1000 r--p    26000       0 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f49ec9b1000     0x7f49ecb07000 r-xp   156000   26000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f49ecb07000     0x7f49ecb5a000 r--p    53000  17c000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f49ecb5a000     0x7f49ecb5e000 r--p     4000  1cf000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f49ecb5e000     0x7f49ecb60000 rw-p     2000  1d3000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f49ecb60000     0x7f49ecb6d000 rw-p     d000       0 [anon_7f49ecb60]
    0x7f49ecb6d000     0x7f49ecb78000 r--p     b000       0 /usr/lib/x86_64-linux-gnu/libgmp.so.10.4.1
    0x7f49ecb78000     0x7f49ecbd5000 r-xp    5d000    b000 /usr/lib/x86_64-linux-gnu/libgmp.so.10.4.1
    0x7f49ecbd5000     0x7f49ecbec000 r--p    17000   68000 /usr/lib/x86_64-linux-gnu/libgmp.so.10.4.1
    0x7f49ecbec000     0x7f49ecbed000 r--p     1000    7f000 /usr/lib/x86_64-linux-gnu/libgmp.so.10.4.1
    0x7f49ecbed000     0x7f49ecbee000 rw-p     1000    80000 /usr/lib/x86_64-linux-gnu/libgmp.so.10.4.1
    0x7f49ecc1e000     0x7f49ecc20000 rw-p     2000       0 [anon_7f49ecc1e]
    0x7f49ecc20000     0x7f49ecc24000 r--p     4000       0 [vvar]
    0x7f49ecc24000     0x7f49ecc26000 r-xp     2000       0 [vdso]
    0x7f49ecc26000     0x7f49ecc27000 r--p     1000       0 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7f49ecc27000     0x7f49ecc4d000 r-xp    26000    1000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7f49ecc4d000     0x7f49ecc57000 r--p     a000   27000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7f49ecc57000     0x7f49ecc59000 r--p     2000   31000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7f49ecc59000     0x7f49ecc5b000 rw-p     2000   33000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7fff3049e000     0x7fff304bf000 rw-p    21000       0 [stack]
```

PIE & ASLR

PIE (Position Independent Executables)

When ***PIE*** is enabled, **the binary** is compiled as *PIC* (Position Independent Code) and thus can be loaded into any memory location upon execution, as it doesn't use any **fixed** addresses hard-coded in its code.

```
$ checksec ./supermario
[*] '/home/doliv/Desktop/supermario'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
$
```

PIE disabled: base address needs to be *0x400000* always

```
$ checksec ./oneshot
[*] '/home/doliv/Desktop/oneshot'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      PIE enabled
$
```

PIE enabled: base address can be different for each run

ASLR (Address Space Layout Randomization)

ASLR is a protection enabled **by default** **at the OS level**.
When enabled, the segments of all the PIE binaries are loaded each time into **random locations**. This makes Return Oriented Programming (***ROP***) attacks much more difficult to execute *reliably*.

Every address can be decomposed as:
address = base address + offset

Example from "SuperMario" (no PIE):
main_addr = **0x401186** = **0x400000** + **0x1186**

- ***ASLR*** **disabled**: base address is ***fixed*** (always the same on each run).
- ***ASLR*** **enabled**: base address gets ***randomized*** on each run.

ASLR & PIE

NB: We usually refer to:

- "***ASLR***" as the randomization of the libraries' base addresses
- "***PIE***" as the randomization of the binary's base address

PIE →

ASLR →

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
             Start                End Perm     Size  Offset File (set vmmap-prefer-relpaths on)
    0x55dc0608b000     0x55dc0608c000 r--p     1000       0 cherry_pie
    0x55dc0608c000     0x55dc0608d000 r-xp     1000    1000 cherry_pie
    0x55dc0608d000     0x55dc0608e000 r--p     1000    2000 cherry_pie
    0x55dc0608e000     0x55dc0608f000 r--p     1000    2000 cherry_pie
    0x55dc0608f000     0x55dc06090000 rw-p     1000    3000 cherry_pie
    0x7f63d671b000     0x7f63d671e000 rw-p     3000       0 [anon_7f63d671b]
    0x7f63d671e000     0x7f63d6744000 r--p    26000       0 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f63d6744000     0x7f63d689a000 r-xp   156000   26000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f63d689a000     0x7f63d68ed000 r--p    53000  17c000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f63d68ed000     0x7f63d68f1000 r--p     4000  1cf000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f63d68f1000     0x7f63d68f3000 rw-p     2000  1d3000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7f63d68f3000     0x7f63d6900000 rw-p     d000       0 [anon_7f63d68f3]
    0x7f63d6930000     0x7f63d6932000 rw-p     2000       0 [anon_7f63d6930]
    0x7f63d6932000     0x7f63d6936000 r--p     4000       0 [vvar]
```

ASLR (Address Space Layout Randomization)

ASLR introduces **partial randomization of the base addresses** of every PIE object loaded (binary & dynamic libraries).

- [24,20]* fixed bits + [28,32]* random bits + **12 constant bits**

All the standard libs have PIE, so the libs base addresses are random

```c
#include <stdio.h>

int main() {
  printf("%p\n", main);
}
```

```
$ gcc -o aslr aslr.c
$ ./aslr
0x555a66a48139
$ ./aslr
0x55b368caa139
$ ./aslr
0x565252d2b139
$ ./aslr
0x5628d0fd6139
$ ./aslr
0x556475f5e139
$
```

0x555a66a48139
0x55b368caa139
0x565252d2b139
0x5628d0fd6139
0x556475f5e139

Those 12 constant bits will be useful later

**architecture + kernel settings dependant, check /proc/sys/vm/mmap_rnd_bits*

How can we bypass ASLR?

We have to find the **base address** each time the binary is executed (which stays the same until the process exits).

There are several ways, applicable in different scenarios:

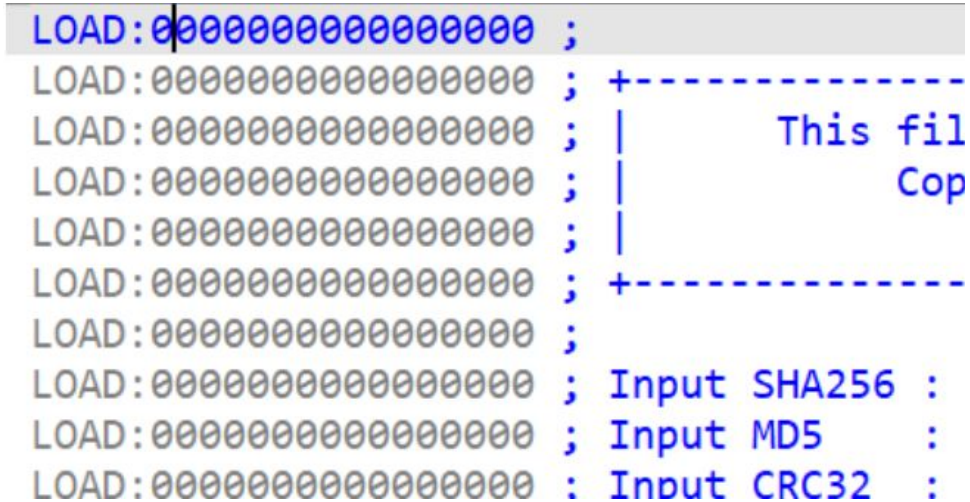
- Generally, the methods used for the canaries do work for leaking ASLR addresses too
- Usually we will find ASLR addresses in the stack
- Format String exploitation
- Strings with no NULL-termination
- Other context-dependent methods

Once we obtained the leaked address we can use it in our exploit.

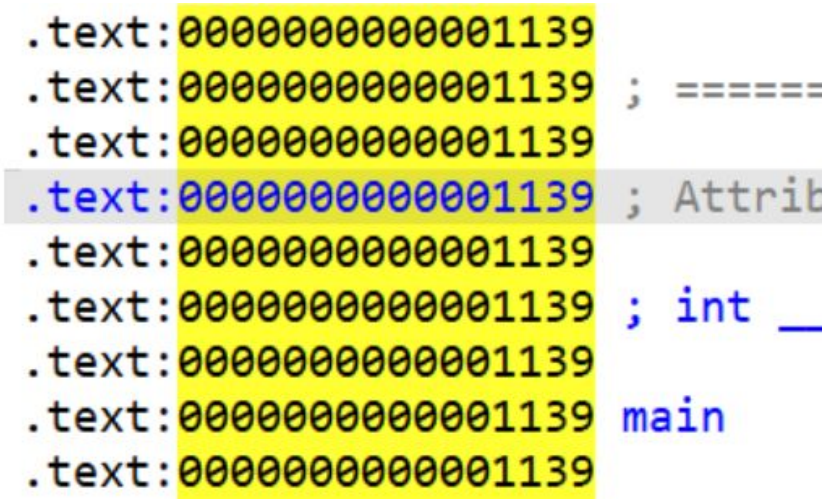
How can we bypass ASLR?

NB: subtract the offset from the leaked address (*GDB is useful*) to obtain the base address.

base_address = address_leak - leak_offest



IDA base: 0x0



Main offset: 0x1139

How can we bypass ASLR?

After obtaining a leak from the program, we can proceed as in the following example:

- Assume we leaked the *main* function address, **0x555a66a48139**
- We know (from a disassembler) its offset from base is: **0x1139**

  0x555a66a48139 - 0x1139 = 0x555a66a47000 = **base address**
- I want to obtain the address of the win function (offset: **0x1293**)

  win address = base address + win offset

  win address = 0x555a66a47000 + 0x1293 = **0x555a66a48293**

NB: Base addresses always end with at least **3 zero nibbles** (0x000)

How can we bypass ASLR?

Sometimes we don't need any leak to jump to a function:

- Assume we can overwrite the return address in a frame
- That return address already contains a randomized address:

  Original return address: 0x555??????139
- We can overwrite just the first few bytes (**1** to **3** to be *reliable*), by doing a **partial overwrite**:

  Win function offset: **0x2478**

  0x555??????139 => 0x555?????X478 (2 bytes overwrite)
- We have to bruteforce only the "X" nibble (**4 bits**, **1/16 chance**)

Tools tricks

Once we have the **base address,** we know every symbol's address in that object (as they share the same base, relative offset is constant).

Here are some tips:

- In **pwndbg**, use "*brva [offset]*" to set breakpoint in PIE binaries
- **Pwntools** has some shortcuts for working with PIE binaries/libs

```
exe = ELF('chall')
base = leak_base(io)
exe.address = base

# will contain the runtime address!
print(exe.sym.main)
```

Setting the base address allows us to use ***exe.sym["func"]*** without any problems by letting pwntools calculate the offset sum automatically (base + sym offset).

LIBC

Get LIBC offset

We have seen how to retrieve a function address in a **PIE** binary, knowing its offset and a leaked address. It works the same way with dynamic libraries (like LIBC): if the binary uses it, the entire LIBC object is loaded in the process memory space, so theoretically **every LIBC function can be executed** if we know its address, but there is a catch:

Do we know the offset of a LIBC function?

- If we are provided with the LIBC binary the server is using, **YES**! (We can find the address of any libc function from a disassembler)
- If we don't have the exact LIBC that our program is using (on the server), it can be trickier. Luckly, there is a tool that can help us!

LIBC database

libc.rip

is a database of all the existing libc versions, based on the **12 least significant constant bits**.
Associating those to the respective functions, it gives you all possible compatible LIBCs with those matches.
Then you can download the exact LIBC and locally get the offset of every function or directly take common offsets from the site.

```
pwndbg> p puts
$1 = {int (const char *)} 0x7ffff7e1fbe0 <__GI__IO_puts>
```

Other similar site: libc.blukat.me

LIBC database with pwntools

Pwntools can automate the search for compatible libcs given (symbol, offset) pairs.

```
192 # get libc leak
193 puts_leak = u64(show_desc()[:8])
194 log.info(f'{puts_leak = :x}')
195
196 # find libc from puts addr
197 libc = ELF(pwnlib.libcdb.search_by_symbol_offsets({'puts': puts_leak}, select_index=10))
198 libc.address = puts_leak - libc.sym['puts']
199 log.info(f'{libc.address = :x}')
```

Once you've found the correct libc between the possible matches, after multiple program launches trying a different one each time, you can fix the choice using the select_index argument

```
138
139
140 # trying to find the remote libc from the offset __libc_start_main+X (X = 133 locally) leaked from stack.
141 for x in (range(-1, 2) if args.LOCAL else range(-1, 30)):
142     # if libc := libcdb.search_by_symbol_offsets({"__libc_start_call_main": libc_leak-122+x}):
143     if libc := libcdb.search_by_symbol_offsets({"__libc_start_main": libc_leak-133+x}):
144         libc = ELF(libc)
145         libc.address = libc_leak-133+x - libc.sym['__libc_start_main']
146         print(libc, hex(libc.address))
147         if input('is that it? ') == 'y':
148             break
149 # the above code found some possible libc versions and libc6_2.35-0ubuntu3.8_amd64 was the correct candidate.
150 print('using', libc, hex(libc.address))
151
```

more desperate case

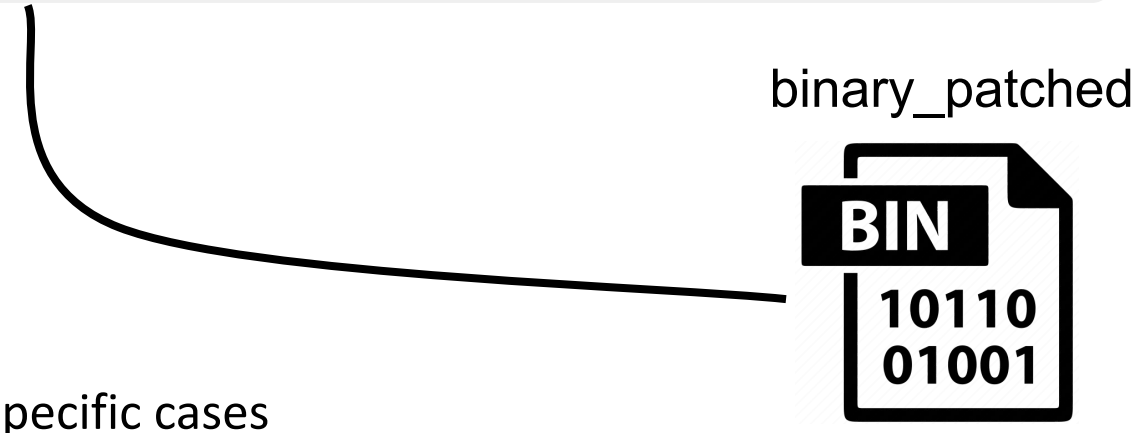
PWNINIT

pwninit is a useful tool when it comes to working with a given libc binary from the server on your system.

It can patch a binary to use any given libc binary, so you can reproduce the exact behavior* of the program in the server.

```
$ ~/tools/pwninit --no-template --bin=binary --libc=libc.so.6 --ld=ld-linux-x86-64.so.2
```

NB: You should also use the same loader of the libc binary if provided.



*other OS settings may vary behaviour in some specific cases

ret2libc

As we said, in a PIE binary we can recover the address of a function from its offset like this:

func_address = base_address + func_offset

This both works in programs and libraries like LIBC, which is always used in dynamically linked binaries.
So we can call any LIBC function, even if it's not present in the **PLT** section (aka, it's not used by the binary).
The same concept applies to ROP gadgets as well.

This technique is called "***ret2libc***"

LIBC tips

Additional tips about **libc**:

- Normal **glibc** binaries always contain a */bin/sh* string.
  You can find its address (or offset if *libc.address* has not been set) using the libc ELF binary with:

  ```
  binsh_addr = next(libc.search(b'/bin/sh\0'))
  ```

- **glibc** always contains lots and lots of code, and lots of **cool gadgets**, useful for every need, and you find them with **ropper** or:

  ```
  libc = ELF('./libc.so.6')
  libc.address = leak_libc_base()
  rop = ROP(libc)
  rop.rcx.address
  ```

PLT & GOT

PLT & GOT

Often a program needs to call an external function. But how can it know where to jump, if ASLR randomizes the address of that function?

When an external function is called, the program first **calls the respective *PLT* function**: here there is an instruction that **jumps to the address stored into the *GOT* entry** of that function.

This entry can be:

- the already resolved address of the function;
- the address of the resolving procedure of that specific called function, which is in the PLT segment too.

PLT & GOT

PLT

```
def printf(...):
    func = GOT["printf"]
    return func(...)

...
```

GOT

```
GOT = {
  "printf": 0x7ff1234567,
  "system": 0x7ff2345678,
  "puts": 0x7ff3456789,
}
```

PLT & GOT

PLT (Procedure Linkage Table)

Executable segment, with:

- the jump instructions that execute the addresses stored in the GOT entries;
- the trampoline procedures that resolves the external function address, write it into its GOT entry, and executes it.

GOT (Global Offset Table)

Maybe-writable segment, with an entry for each external function, which can be:

- the actual address of the function (address in the library memory map);
- the address of the trampoline procedure in the PLT segment (address in the binary memory map).

In both cases, this maybe-writable segment contains addresses that will be executed!

Getting leaks from GOT

gets@got is at *0x404038*

Calling puts(0x404038) will print the address stored in the *gets* got entry.
It can be:

- pointer to the trampoline procedure that resolves the gets address (pointer to *gets@PLT*) if *gets* has not been called (hence resolved) yet.
- the actual resolved address of the *gets* function (in the form *libc_base + gets_offset)*

```
.got.plt:0000000000404000 ; ========================================
.got.plt:0000000000404000
.got.plt:0000000000404000 ; Segment type: Pure data
.got.plt:0000000000404000 ; Segment permissions: Read/Write
.got.plt:0000000000404000 _got_plt        segment qword public 'DA
.got.plt:0000000000404000                 assume cs:_got_plt
.got.plt:0000000000404000                 ;org 404000h
.got.plt:0000000000404000 _GLOBAL_OFFSET_TABLE_ dq offset _DYNAMIC
.got.plt:0000000000404008 qword_404008    dq 0
.got.plt:0000000000404010 qword_404010    dq 0
.got.plt:0000000000404018 off_404018      dq offset puts
.got.plt:0000000000404020 off_404020      dq offset fread
.got.plt:0000000000404028 off_404028      dq offset setbuf
.got.plt:0000000000404030 off_404030      dq offset printf
.got.plt:0000000000404038 off_404038      dq offset gets
.got.plt:0000000000404040 off_404040      dq offset malloc
.got.plt:0000000000404048 off_404048      dq offset fopen
.got.plt:0000000000404048 _got_plt        ends
.got.plt:0000000000404048
.data:0000000000404050 ; ========================================
```

PLT & GOT

PLT

```
def gets(...):
  func = GOT["gets"]
  return func(...)

def resolve_gets(...):
  GOT["gets"] = dl_resolve("gets")
  return GOT["gets"](...)
```

GOT

```
GOT = {
  "printf": 0x7ff1234567,
  "system": 0x7ff2345678,
  "puts": 0x7ff3456789,
  "gets": resolve_gets,
}
```

RelRO

The ***RelRO*** **(Relocation Read-Only)** level at which the binary was compiled establishes if the GOT is **writable** or **read-only**:

- **No *RelRO* / Partial *RelRO*:** with this protection, the binary resolves the external functions at runtime, so the first time an external function needs to be executed, the program resolves its address, storing it in the *GOT*. This makes the *GOT* **writable**!

- **Full *RelRO*:** with this protection, the loader resolves all the external functions of the binary at the start of the program, and after that makes the *GOT* read-only.

Multistage

ret2main

Imagine to have only one vulnerability in the program, executed one time, that consents you to **alter the execution flow** (like ROP); but you haven't any libc leaks, and there is no win function in the binary. How can we exploit it?

It would be great if we could **execute again the vulnerability**…

We can use the vulnerability to give us some leaks, and then **jump to the main function to run again the same vulnerability**, but this time using the previously obtained leaks, and without restarting completely the program, so the addresses are the same!

The same thing is works with the **_start** symbol and is called **ret2start**

Multistage Exploits

The flow of a multistage exploit will usually be:

- Leak values from the **GOT** (or in any other way),
  usually through a ROP doing *puts* or through a format string vuln
- Calculate the libc base address

  libc = ELF('./libc.so.6')

  libc.address = gets_got_leak - libc.sym.gets
- Use the obtained libc addresses doing ROP again

  libc.sym.system

Stack Pivot

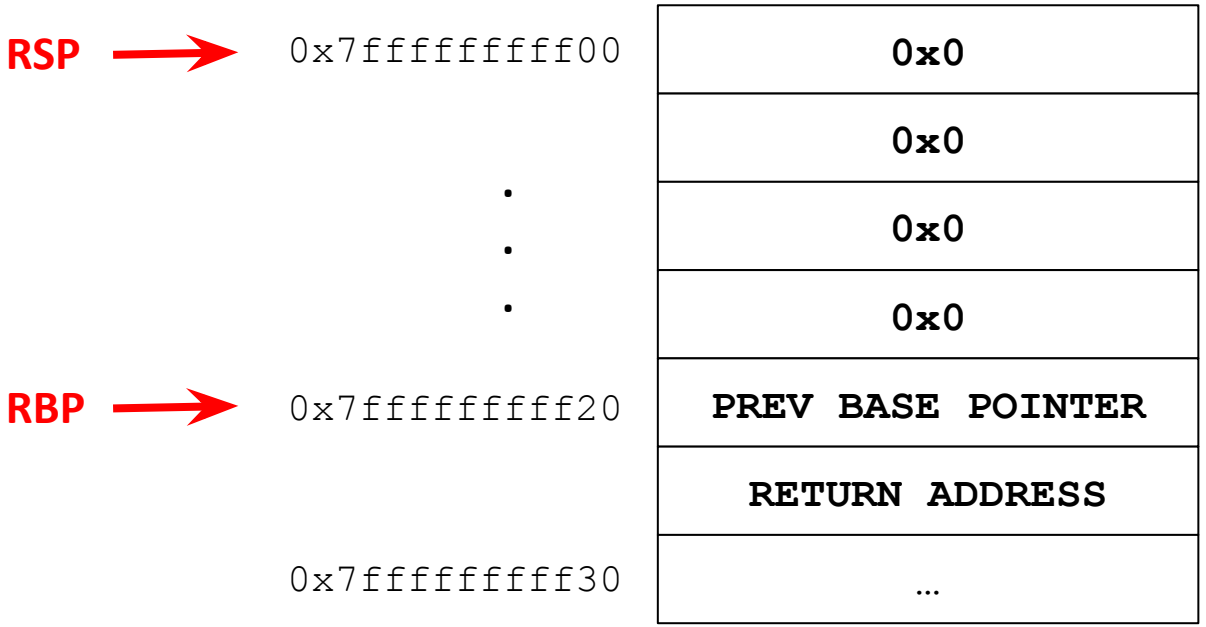
Stack pivot

The values of the registers **RBP and RSP can be *anything***, they aren't restricted to be a stack address. Having a buffer overflow, we can exploit that:

```
char data[0x100] = {0};

int main() {
  char buf[32] = {0};
  gets(buf);
  return 0;
}
```

RIP →



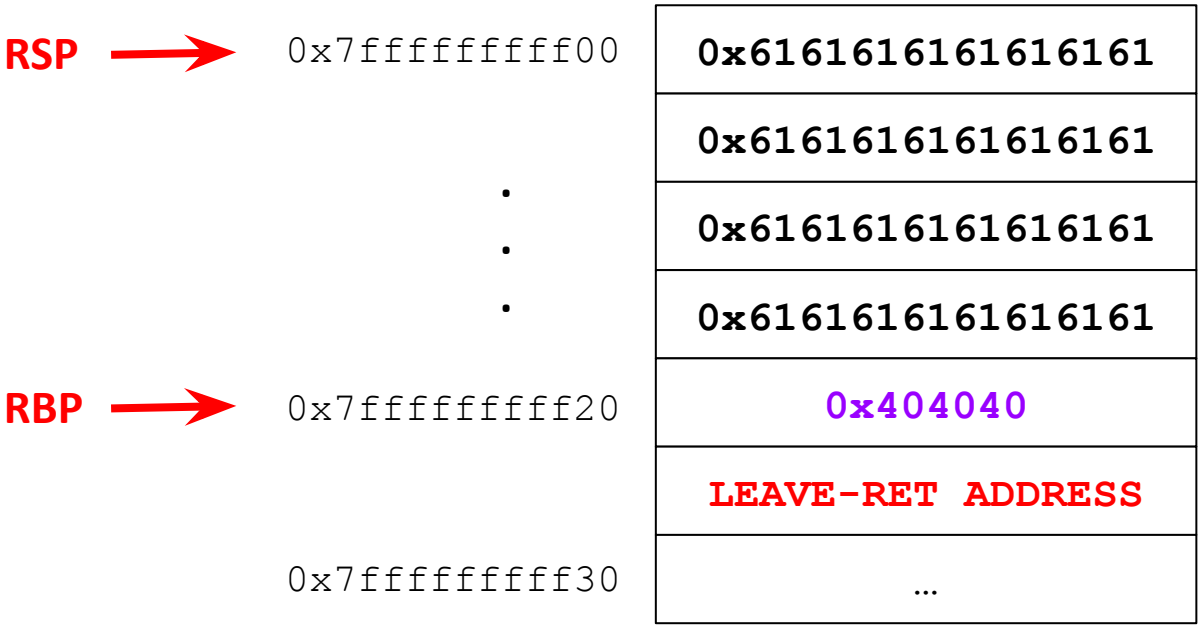
Stack pivot

The values of the registers **RBP and RSP can be *anything***, they aren't restricted to be a stack address. Having a buffer overflow, we can exploit that:

```
char data[0x100] = {0};

int main() {
  char buf[32] = {0};
  gets(buf);
  return 0;
}
```

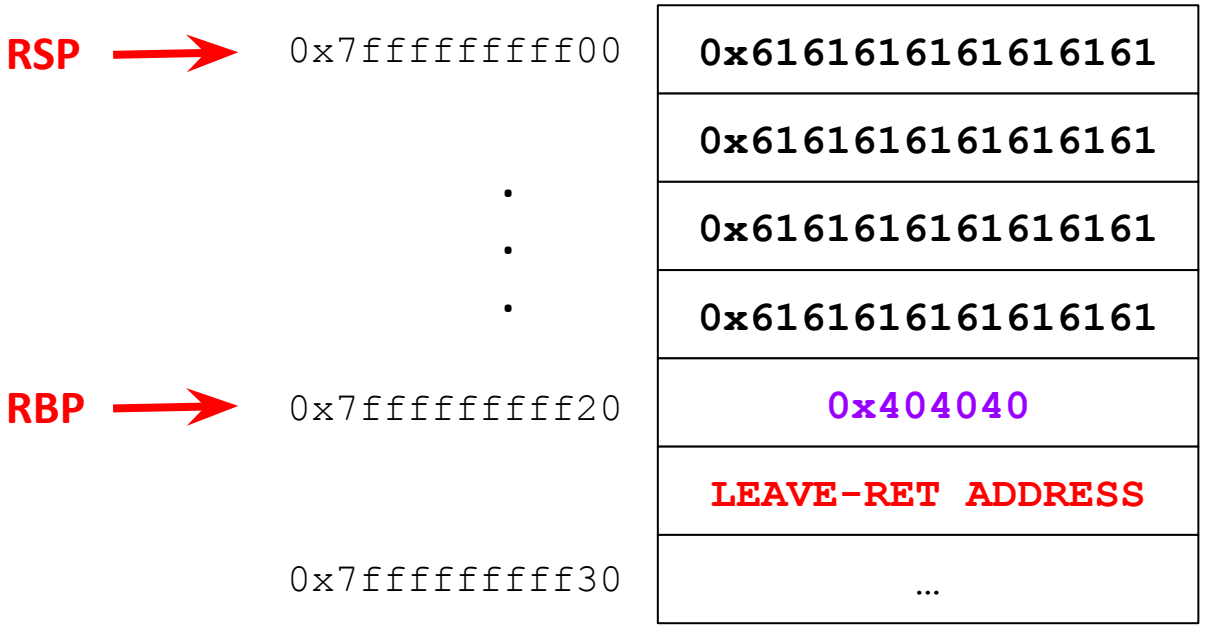
RIP →



Stack pivot

The values of the registers **RBP and RSP can be *anything***, they aren't restricted to be a stack address. Having a buffer overflow, we can exploit that:

NB: *leave = mov rsp, rbp; pop rbp*

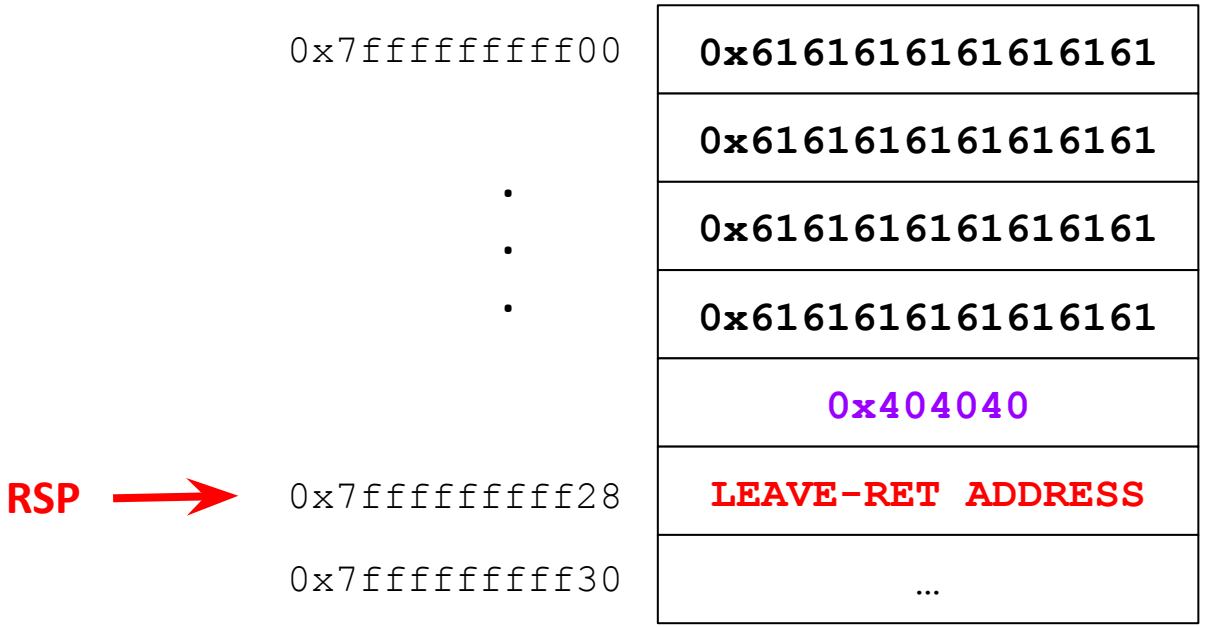


Stack pivot

The values of the registers **RBP and RSP can be *anything***, they aren't restricted to be a stack address. Having a buffer overflow, we can exploit that:

NB: *leave = mov rsp, rbp; pop rbp*

RBP = 0x404040

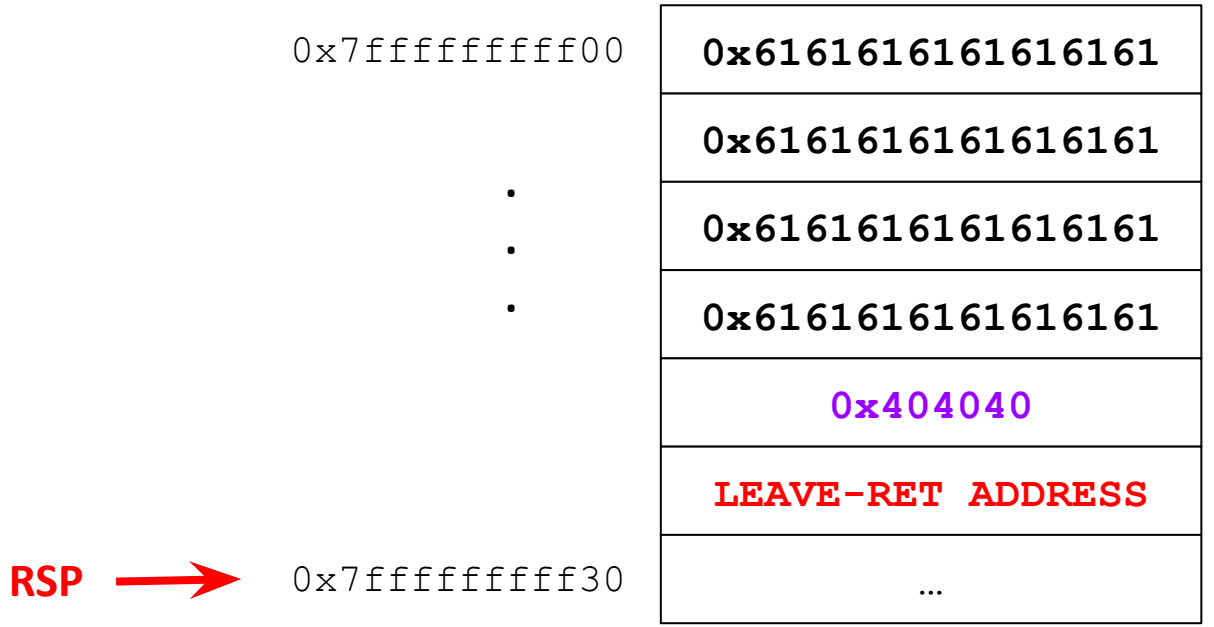


Stack pivot

The values of the registers **RBP and RSP can be *anything***, they aren't restricted to be a stack address. Having a buffer overflow, we can exploit that:

NB: *leave = mov rsp, rbp; pop rbp*

RBP = 0x404040



Stack pivot

The values of the registers **RBP and RSP can be *anything***, they aren't restricted to be a stack address. Having a buffer overflow, we can exploit that:

NB: *leave = mov rsp, rbp; pop rbp*

RBP = *0x404040

RSP = 0x404048

0x7fffffffff00	0x6161616161616161
	0x6161616161616161
.	0x6161616161616161
.	0x6161616161616161
.	0x6161616161616161
	0x404040
	LEAVE-RET ADDRESS
0x7fffffffff30	...

*0xDEADBEEF : What is stored at the address 0xDEADBEEF

Stack pivot

The values of the registers **RBP and RSP can be *anything***, they aren't restricted to be a stack address. Having a buffer overflow, we can exploit that:

NB: *leave = mov rsp, rbp; pop rbp*

RBP = *0x404040

RSP = 0x404050

RIP = *0x404048

0x7fffffffff00	0x6161616161616161
	0x6161616161616161
	0x6161616161616161
	0x6161616161616161
	0x404040
	LEAVE-RET ADDRESS
0x7fffffffff30	...

*0xDEADBEEF : What is stored at the address 0xDEADBEEF

Some useful links

Syscalls reference sites

- *https://x64.syscall.sh/*
- *https://x86.syscall.sh/*
- *https://blog.rchapman.org/*
- *https://syscalls.mebeim.net*

Online compilers/assemblers

- https://defuse.ca/
- *https://disasm.x32.dev/*
- *https://godbolt.org/*

Cool PWN notes

- https://ir0nstone.gitbook.io/
- *https://github.com/Naetw/CTF-pwn-tips*
- *https://ctf-wiki.mahaloz.re/*

Additional tools

- https://cutter.re
- *https://github.com/david942j/one_gadget*

Challenges

1: Arbitrary 1

You have a single write

2: Arbitrary 2

You have a read and a write

3: Severe SAM

Since you have pwned also the new version of our software, we have updated it again, this time will be sweet like a *pie*…

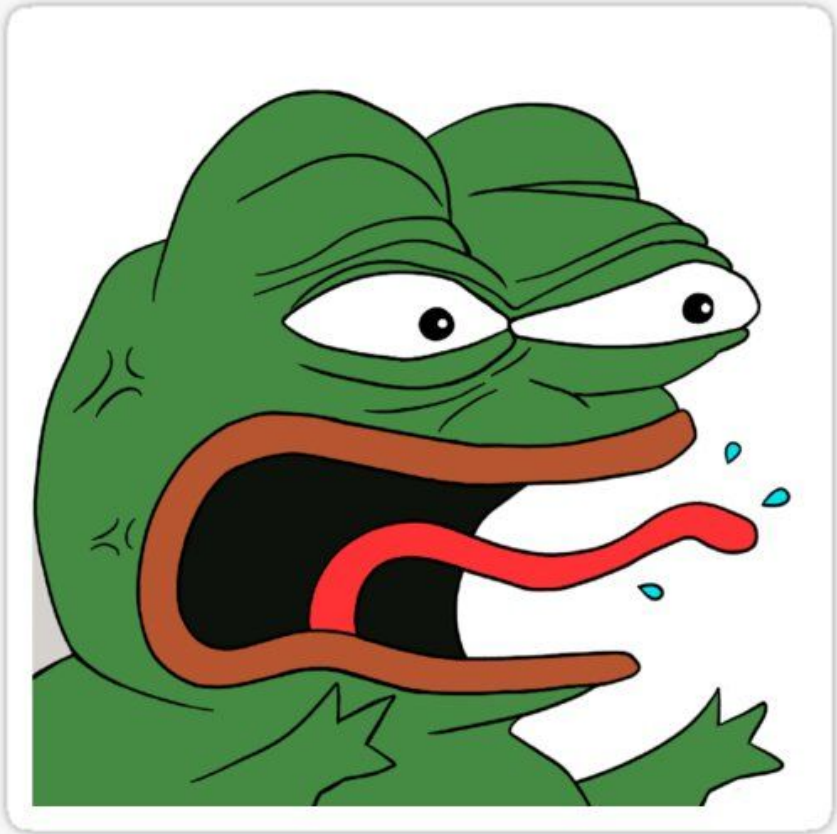
4: Cherry Pie

We all love cherry pies, are you able to get the base address, while take down bird that wants to eat it?

5: Angry Pepe

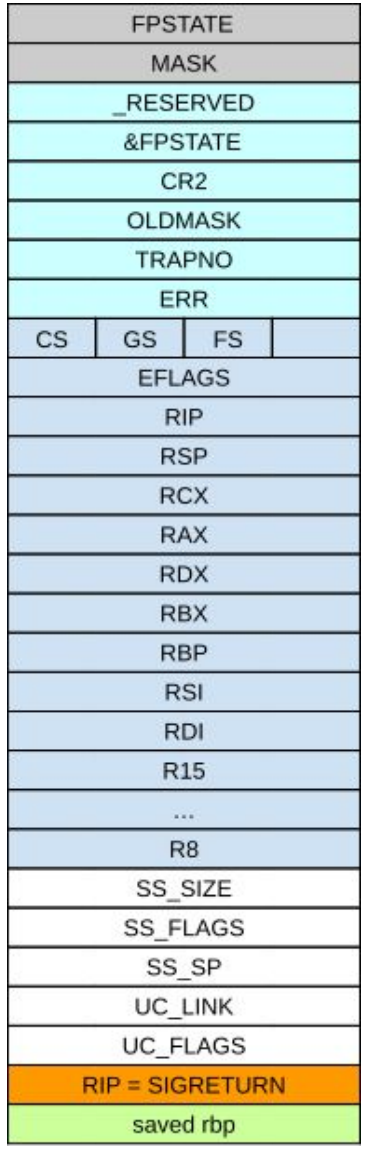
Pepe is angry because you pwn his service and won't use system anymore. You will be able to pwn this?

Check well all the code inside this program…



5: ezSigRop

- This program does literally nothing
- Almost no gadget for you
- How are you gonna get a shell?



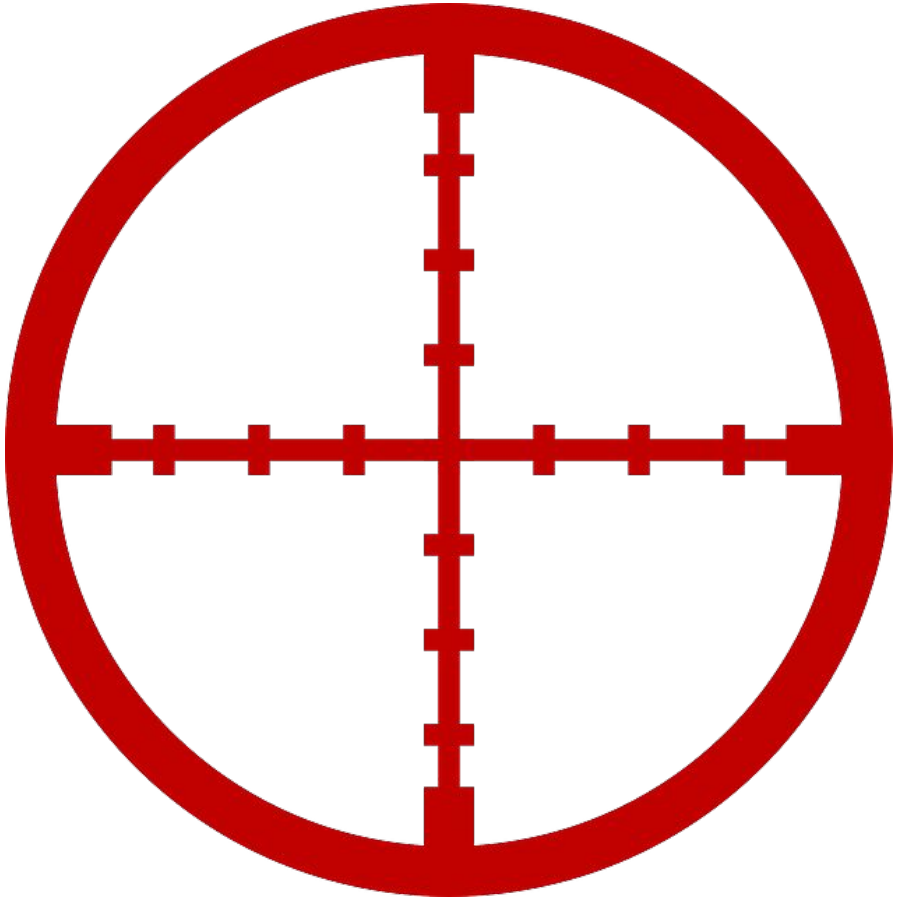
6: Super Mario Odyssey

This time Mario won't been able to find the Castle! Can you help him?

7: Traveller

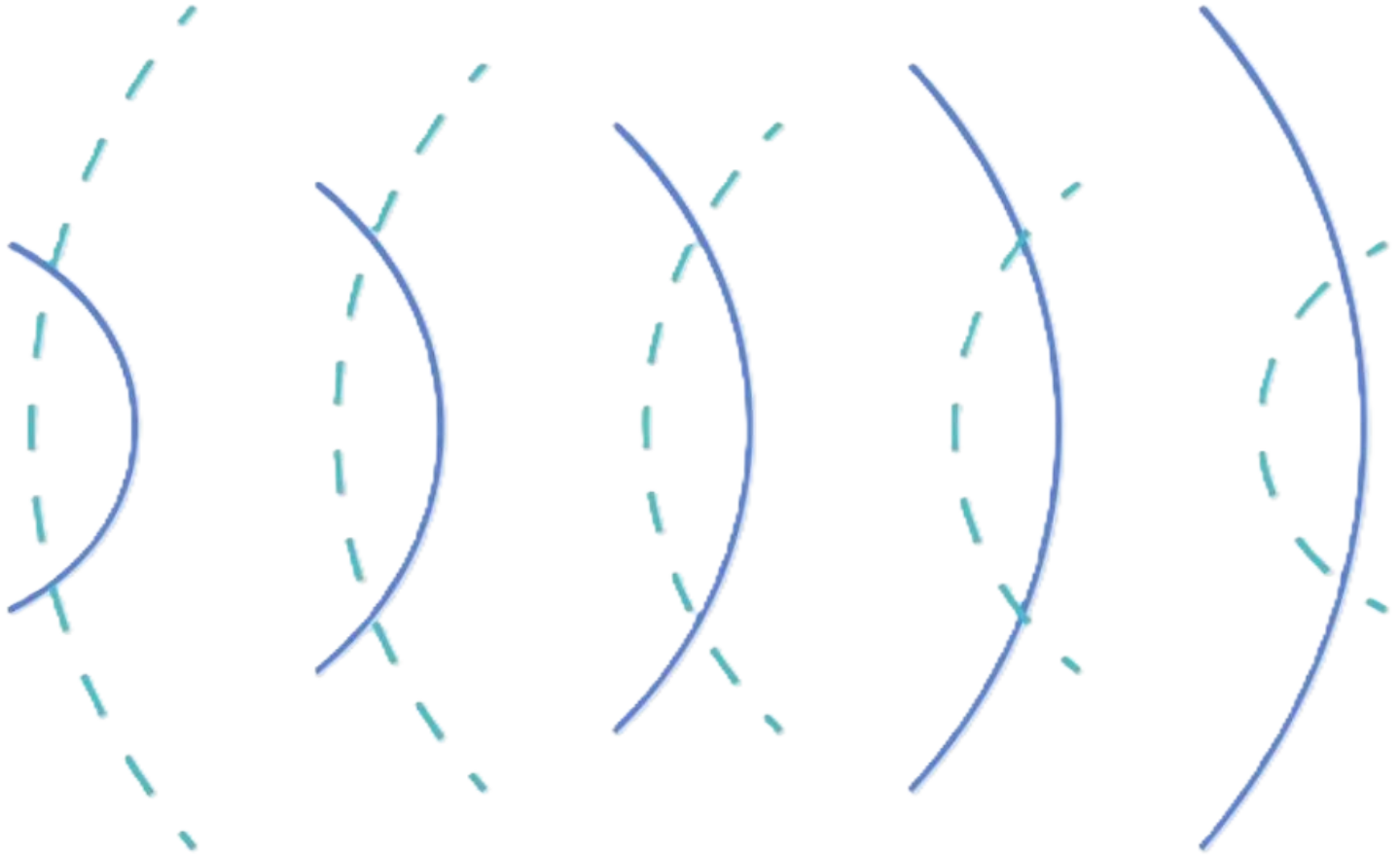
- Humans as a species have the know-how required to travel to far-away places with relative ease, bringing their entire belongings with them.
- Prove your humanity by making use of such know-how.

8: One Shot



9: Echo

The voices…
In my head…
Won’t… Go away…



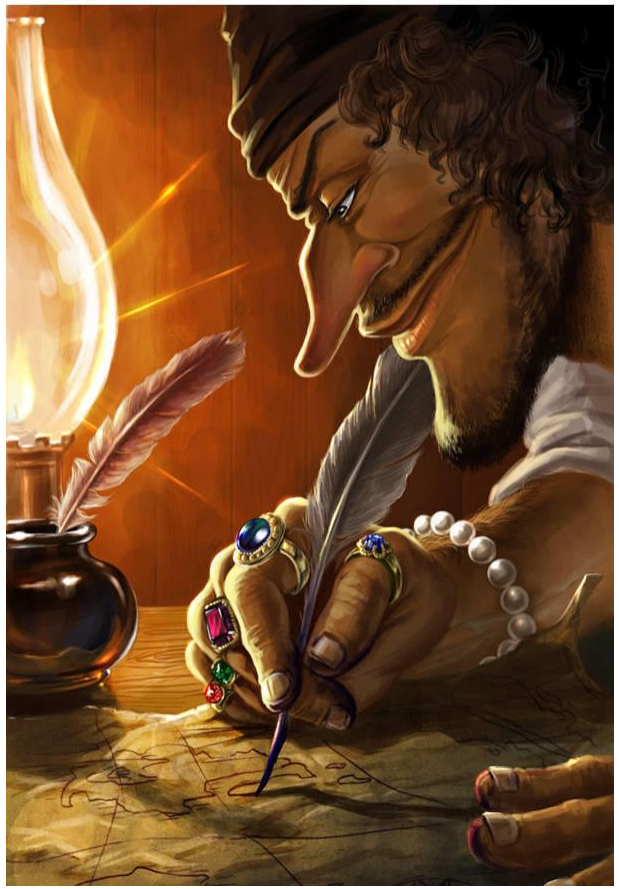
10: Pie Shop

The best pie shop you've ever seen

10: COOKB00K

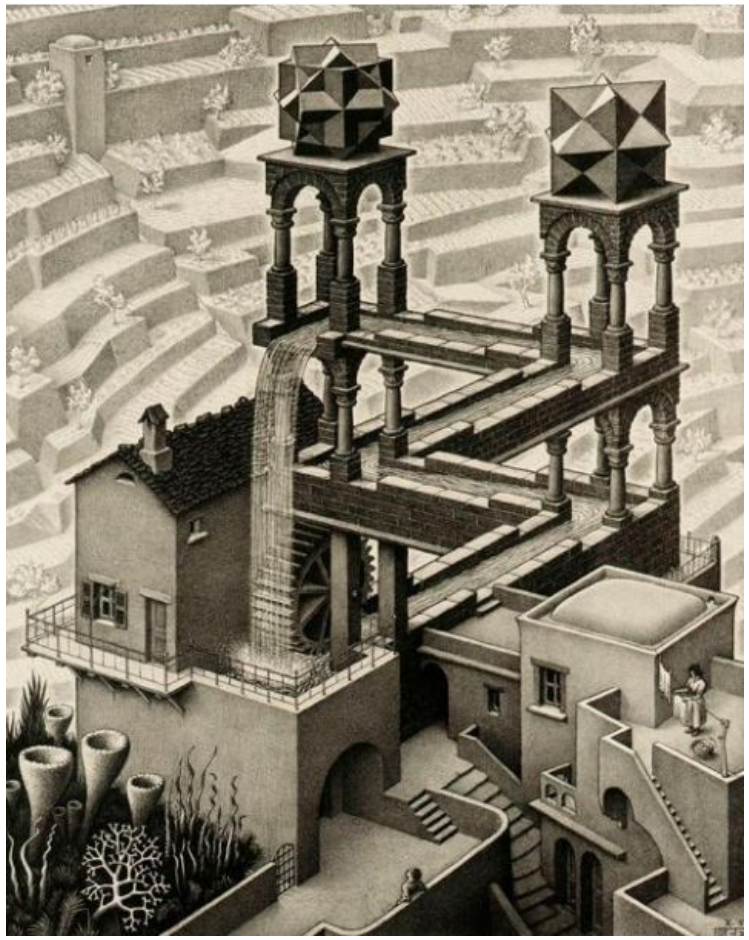
Are you a good chef?
Prove it by writing down all your recipes in my cookb00k!

HOW IT FEELS TO COOK THE PERFECT ROP TO RUN /bin/sh



B3: Brainfuck

- Brainfuck is a turing-complete language.
- Modelled after the theoretical model of a Turing Machine.
- It is good to keep the model of the brainfuck turing machine in mind



N3: Demon

- The monks have been sensing the eerie presence of a great Demon hiding in their temple.
- However, no one has yet been able to find this evil being, much less destroy it.
- Whoever seeks to slay it is required to have perfect clarity of mind and utter confidence in their arts, or catastrophic failure will certainly follow.